
THE AUSTRALIAN NATIONAL UNIVERSITY

Second Semester Final Examination – November 5th, 2020

COMP3320/6464/HONS HIGH PERFORMANCE SCIENTIFIC COMPUTATION

- Study period: 15 minutes
- Time allowed: 3 hours and 45 minutes (including submission time to Wattle)
- Permitted materials: open-book exam
- Exam questions total 100 marks
- All students answer all questions
- Clarity and conciseness in answers will be highly valued
- Marks may be lost for supplying irrelevant information

1. Short Questions [20 marks]

a) Each of the following statements are either true or false. Which are they? Give a few sentences to explain each of your answers.

- i. The relative error of a division operation is approximately the sum of the relative errors of the numerator with the relative error of the denominator.

[2 marks]

- ii. In floating-point arithmetic, the machine precision is the smallest positive number that perturbs the number 0 when added to it.

[2 marks]

- iii. If two real numbers are exactly representable as floating-point numbers, then the result of *any* real arithmetic operation on them will also be an exactly representable floating-point number, provided that the result is not larger/smaller than the overflow/underflow limit.

[2 marks]

b) Define the term cancellation error. How does this relate to the machine precision?

[2 marks]

c) For the following Pthread program

```
#define N 2
double *ptr1, *ptr2;
sem_t mutex;

int main(){
    pthread_t TID[N];
    double carray[2] = {0.27,0.73};
    double varray[4] = {1,2,3,4};
    ptr1 = carray;
    ptr2 = varray;
    sem_init(&mutex, 0, 1);
    for(int i = 0; i < N; ++i)
        Pthread_create(&TID[i], NULL, thread, (void *)i);
    for(int i = 0; i < N; ++i)
        Pthread_join(TID[i], NULL);
    exit(0);
}

void *thread(void *vargp){
    int myID = (int)vargp;
    static int sum = 0;
    P(&mutex);
    sum += ptr1[myID]*(ptr2[myID]+ptr2[myID+2]);
    V(&mutex);
    return NULL;
}
```

discuss how each of the variables `carray`, `varray`, `i`, `myID`, `sum`, `ptr1` and `ptr2` is mapped to memory, how many instances of each variable there are, and which ones are shared variables according to the Pthread memory model.

[4 marks]

d) Do cache coherence protocols obviate the need for locks? Explain your answer in detail.

[2 marks]

e) According to Dennard scaling, if transistor feature size is scaled down by a factor of $(1/k)$, one can increase the number of transistors by a factor of k^3 and keep the power consumption constant. Demonstrate whether the previous statement is correct. Prove whether the previous statement is consistent with Dennard and whether it explains the exponential increase in performance during the uniprocessor era.

[4 marks]

f) What are the main similarities and differences between the Pthread and OpenMP memory and execution models?

[2 marks]

2. Shared Memory Programming with OpenMP [20 marks]

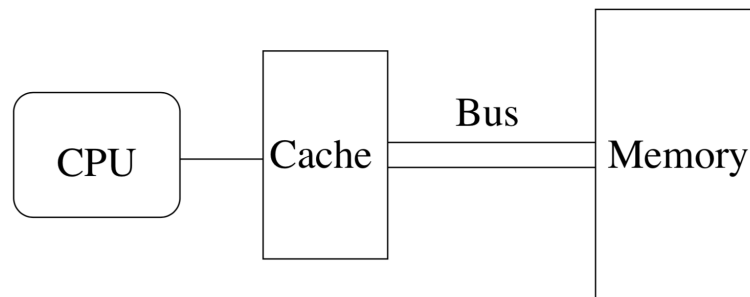
The following C routine was found, by profiling, to dominate the computation time in a larger program. Your colleague has attempted to parallelize the routine using OpenMP. For the purpose of this question you should assume that there is no pointer aliasing.

```
void mystery(double f[], double d[], double val[], int n){
    int i,j,k,l,ij,kl,ixf;
    ij = 0;
    for(i = 0; i < n; ++i){
        for(j = 0; j <= i; ++j){
            ++ij;
            f[i*(i+1)/2+j] = 0.0;
            kl = 0;
#pragma omp parallel default(shared)
            for(k = 0; k < n; ++k){
#pragma omp for schedule(static,10)
                for(l = 0; l <= k; ++l){
                    ++kl;
                    ixf = i*(i+1)/2+j;
                    f[ixf] += d[k*(k+1)/2+l]*val[ij*(ij+1)/2+kl];
/* question requests values of ij, kl and ixf if printed here */
                }
            }
        }
    }
}
```

- What would be the values of `ij`, `kl` and `ixf` at the point indicated by the comment line – assuming that the code is running in serial mode – when `i=5`, `j=4`, `k=3`, `l=2`?
[4 marks]
- How does the calculation scale with the parameter `n`? Provide an analytic expression in terms of `n`.
[4 marks]
- How many unique elements of arrays `f`, `d` and `val` are accessed? Provide an analytic expression in terms of `n`.
[4 marks]
- The code when compiled with OpenMP enabled and run in parallel on multiple threads gives incorrect results. Explain why this is the case by discussing each error.
[6 marks]
- Rewrite the routine so that it runs correctly in parallel using OpenMP and with high efficiency. You are free to change the code as you like, adding additional local variables if required. Explain briefly why you performed each optimization. The effects on arrays `f`, `d` and `val` must remain the same.
[6 marks]

3. High Performance Computer Architecture [20 marks]

Consider the following memory system



- The cache line size is 64 bytes.
- One word is 64 bits.
- The cache uses a no-write-allocate policy.
- The cache is write-through.
- The bus can transfer data from and to memory at a rate of 32 GB/s (only in one direction at a given time).

The following behaviour is observed when running a certain application:

- The CPU sends memory transaction requests (loads or stores of one word) to the cache at a rate of 4×10^9 requests/sec, with 35% of the requests being writes.
- 96% of all memory requests issued by the CPU are resolved by data already in cache (i.e. the cache hit ratio is 96%).

a) Given the above, compute the bus traffic generated (in GB/s) by the following events:

- Read hits.
- Read misses.
- Write hits.
- Write misses.

[4 marks]

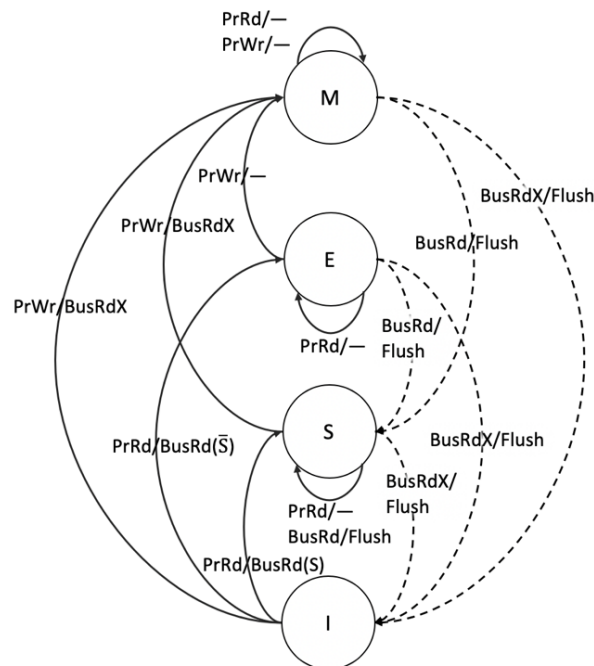
b) What percentage of the total available bus bandwidth is used?

[4 marks]

c) What additional information do you require in order to estimate the percentage of the total available bus bandwidth used if the cache was using a write-back policy?

[4 marks]

- d) Your machine implements (in hardware) the MESI cache coherence protocol shown in the following state diagram



The table below describes the behaviour of a program running in parallel using two processors P1 and P2 on your machine. C1 and C2 are cache lines associated with processors P1 and P2. The notation &X indicates the address of variable X, while mem[&X] is the content of the memory cell (*i.e.* the value of X) at address &X. Note that the “bus transactions” are initiated as a consequence of the processor action and they correspond to the solid lines in the protocol (processor action/bus transaction), while the bus snoopers transactions correspond to the dashed lines.

Following the MESI protocol, fill the entries in the table based on the listed processor actions.

processor action	P1 cache miss/hit, X value	P2 cache miss/hit, X value	Bus transaction (processor action initiated)	Bus snoopers transaction	C1 state	C2 state	mem[&X]
No-Op	N/A	N/A	—	—	I	I	0
P1Rd X	miss, 0	N/A	BusRd($\bar{S}$)	—			
P1Wr X = 2							
P2Rd X							
P1Wr X = X-1							
P2Wr X = X+1							

[8 marks]

4. High Performance Computer Architecture [20 marks]

The C pseudo-code below computes a matrix vector product of a $M \times N$ matrix (`mat`) with a vector (`vin`) of length N . The code assumes that the elements of the array `vout`, which contains components of the output vector, are initialized to zero.

```
for(int i = 0; i < M; ++i){
    for(int j = 0; j < N; ++j){
        vout[i] += mat[i][j]*vin[j];
    }
}
```

- a) Outline how you would write a routine to implement this operation using SSE2 instructions. You are not required to reproduce the SSE2 syntax exactly, but you must make your intention clear. Assume single-precision floating-point data type and explicitly write how you would allocate and structure the `vout`, `mat`, and `vin` arrays for aligned access and best vector performance.

[8 marks]

- b) Write a routine to implement this operation using OpenMP SIMD directives.

[4 marks]

- c) Assume now that in your OpenMP program the `vout` and `vin` arrays are 8-byte aligned. Discuss how the compiler may vectorize your code (there is more than one case), explicitly accounting for cases when N or $N-1$ is divisible by the SIMD vector length.

[6 marks]

- d) The following C routine

```
void shifted_average(double *a, double *b, int n){
    for(int i = 1; i < n; ++i){
        b[i] = 0.5*(a[i]+a[i-1]);
    }
}
```

is unlikely to be vectorized by the compiler. Explain why and provide two different ways of forcing the compiler to vectorize this code (without using explicitly vector instructions such as SSE).

[2 marks]

5. Performance Modelling [20 marks]

The weak speedup of an OpenMP parallel code running on N threads, each pinned to a distinct CPU core, can be modelled as

$$S_w(N) = \frac{s + (1 - s) N^\alpha}{s + (1 - s) N^{\alpha-1} + c(N)}$$

where $c(N)$ is a continuous and differentiable function of N and s is a constant associated with the serial fraction of the application code.

- a) Discuss the meaning of the $c(N)$ function and derive the strong speedup expression $S_s(N)$.

[2 marks]

- b) Assuming $c(N) = \gamma N$, derive an analytic expression for the maximum strong speedup achievable $S_s^{max} \equiv S_s(N = N_{max})$ and the number of threads N_{max} that yields this speedup.

[8 marks]

- c) For another application code we observe that the serial fraction of the code changes as a function of the number of threads N (when running in parallel). The strong speedup of the code can now be modelled as

$$S_s(N) = \frac{1}{s_p(N) + p/N + \gamma}$$

where $s_p(N) = s f(N)$ is an expression associated with serial fraction of the code when running in parallel, s is a constant associated with the serial fraction of the code when running in serial ($N = 1$), and p is a constant associated with the parallel fraction of the application code. Note that s and p are constants and do not vary with the number of threads.

Derive a *simple* expression for the $f(N)$ function such that superlinear strong speedup is achieved for a range of N values, subject to the constraint that $\lim_{N \rightarrow \infty} S_s(N) = 0$ (provided that $\gamma > 0$).

[4 marks]

- d) By performing a weak scaling analysis for your application code with $\alpha = 1$ and $s = 0.07$ your colleague obtained the following weak speedup data

S_w	N (number of threads)
1.8558	2
2.6238	3
3.2672	4

3.7760	5
4.1544	6
4.4161	7
4.5793	8
4.6630	9
4.6850	10

Assuming that $c(N) = \gamma N^\beta$, calculate the γ and β parameters and discuss their relationship with the extent of parallel communication overhead in your application code.

[6 marks]
